

Part I: Short Answer / Reasoning Questions

Q1. Counting NULL values

Explanation: The query returns 0 because NULL represents an unknown or missing value. In SQL, evaluating any value against NULL using the = operator results in UNKNOWN, not TRUE, meaning those rows are automatically filtered out.

Corrected Query:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

General exclusion: NULL is excluded from most aggregate computations (like AVG or SUM) and comparison operations because you cannot logically perform math or exact logical comparisons on an undefined and unknown state.

Q2. DISTINCT and GROUP BY

Correctness: Yes, the student is entirely correct.

Justification: The GROUP BY department clause already aggregates the result set so that every unique department appears exactly once in the final output. Applying DISTINCT on top of grouped data is redundant because the resulting rows are inherently distinct by definition of the grouping.

Q3. INNER JOIN vs. LEFT JOIN

Data implications: The difference in row count indicates that there are customers in the Customer table who have not placed any orders.

Scenario: An INNER JOIN only returns records where there is a match in both tables, whereas a LEFT JOIN returns all records from the left table (Customer), filling in NULL values for the right table (Order) if no match exists. Therefore, customers without orders increase the row count for the LEFT JOIN.

Q4. Invalid Standard SQL Aggregation

Correction:

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department, employee_name;
```

Conceptual issue: Standard SQL requires all unaggregated columns in the SELECT clause to be included in the GROUP BY clause. If SQL allowed this query to run as-is, it would not know which specific employee_name to display alongside the department's average salary, since there are multiple employees per department, leading to arbitrary data output.

Q5. Filtering Aggregates

Error and Execution Order: The query fails because the WHERE clause filters individual rows *before* aggregation occurs. It is impossible to use WHERE to evaluate AVG(salary) because the average has not been computed yet.

Corrected Query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. Subquery Types

(a) **Overall average:** A **non-correlated subquery** is more appropriate. The overall average product price is a single, static value that can be computed just once and then

compared against each product.

(b) Department average: A **correlated subquery** is more appropriate. The average salary must be calculated dynamically on a per-department basis, meaning the subquery relies on the department ID of the specific row currently being evaluated in the outer query.

Performance Implication: A correlated subquery executes repeatedly for *every single row* evaluated by the outer query, severely impacting performance on large tables. A non-correlated subquery executes only once.

Part II: Long Answer Question

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Justification: I utilized a LEFT JOIN between Customer and Order and explicitly filtered for WHERE o.order_id IS NULL to safely and accurately isolate customers who completely lack matching records in the order table.

(b) Average order value for each restaurant (> 5 orders)

```
SELECT o.restaurant_id, AVG(order_totals.total_value) AS
    avg_order_value
FROM "Order" o
JOIN (
    SELECT order_id, SUM(mi.price * oi.quantity) AS total_value
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY order_id
) order_totals ON o.order_id = order_totals.order_id
GROUP BY o.restaurant_id
HAVING COUNT(o.order_id) > 5;
```

Justification: I used an inner subquery to first calculate the total value of each distinct order, then joined it back to aggregate averages and enforced the strict > 5 requirement using a HAVING clause after grouping.

(c) Restaurants with above platform-average order values

```
WITH RestaurantAvg AS (
    SELECT o.restaurant_id, AVG(order_totals.total_value) AS
        avg_order_value
    FROM "Order" o
    JOIN (
        SELECT order_id, SUM(mi.price * oi.quantity) AS
            total_value
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY order_id
    ) order_totals ON o.order_id = order_totals.order_id
    GROUP BY o.restaurant_id
```

```

        HAVING COUNT(o.order_id) > 5
    )
    SELECT restaurant_id, avg_order_value
    FROM RestaurantAvg
    WHERE avg_order_value > (
        SELECT AVG(total_value)
        FROM (
            SELECT SUM(mi.price * oi.quantity) AS total_value
            FROM OrderItem oi
            JOIN MenuItem mi ON oi.item_id = mi.item_id
            GROUP BY order_id
        ) platform_totals
    );

```

Justification: I used a non-correlated subquery because the platform-wide average order value is universally static; it does not change based on the restaurant being evaluated and only needs to be computed once.

(d) Most ordered menu item for each restaurant

```

WITH ItemQuantities AS (
    SELECT mi.restaurant_id, mi.item_id, mi.item_name, SUM(oi.
        quantity) AS total_qty
    FROM MenuItem mi
    JOIN OrderItem oi ON mi.item_id = oi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
),
RankedItems AS (
    SELECT restaurant_id, item_id, item_name, total_qty,
        RANK() OVER (PARTITION BY restaurant_id ORDER BY
            total_qty DESC) as rank
    FROM ItemQuantities
)
SELECT restaurant_id, item_id, item_name, total_qty
FROM RankedItems
WHERE rank = 1;

```

Justification: Using GROUP BY paired with MAX() is limited because it calculates the highest quantity but strips away the context, making it extremely difficult to retrieve the associated item_id or item_name alongside the maximum value. Window functions (RANK() OVER) seamlessly bypass this top-N per group limitation.

Part III: Normalization

Q1. Functional Dependencies

- student_id → student_name
- course_id → course_name, instructor_id
- instructor_id → instructor_name, instructor_office
- (student_id, course_id) → grade, enrollment_date

Q2. First Normal Form (1NF)

Yes, the table is in 1NF. Assuming standard entry practices, all columns hold atomic (indivisible) values, and there are no repeating groups or arrays within a single record.

Q3. Second Normal Form (2NF)

- **Partial Dependency:** $\text{student_id} \rightarrow \text{student_name}$ and $\text{course_id} \rightarrow \text{course_name}$, instructor_id . The primary key is composite (**student_id**, **course_id**), but these attributes depend on only *part* of that key.
- **Anomaly Example:** An **insertion anomaly** occurs if an administrator attempts to add a new student into the system who has not yet enrolled in any courses. Because **course_id** is part of the primary key, it cannot be NULL, making it physically impossible to insert the student's name into the database.

Q4. Third Normal Form (3NF)

- **Transitive Dependency:** $\text{course_id} \rightarrow \text{instructor_id} \rightarrow \text{instructor_office}$ and instructor_name .
- **Anomaly Example:** An **update anomaly** occurs if an instructor relocates to a new office. The system would be forced to update the **instructor_office** string on hundreds of individual enrollment rows; missing even one row leads to corrupted, inconsistent data.

Q5. 3NF Decomposition

- Student (**student_id**, student_name)
Justification: Separated to resolve the partial dependency $\text{student_id} \rightarrow \text{student_name}$.
- Course (**course_id**, course_name, instructor_id)
Justification: Separated to resolve the partial dependency $\text{course_id} \rightarrow \text{course_name}$, instructor_id .
- Instructor (**instructor_id**, instructor_name, instructor_office)
Justification: Separated to resolve the transitive dependency $\text{instructor_id} \rightarrow \text{instructor_name}$, instructor_office .
- Enrollment (**student_id**, **course_id**, grade, enrollment_date)
Justification: Maintained to represent the full functional dependency $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$.

Part IV: Design Justification

For query (a), I could have used a `NOT IN` subquery instead of a `LEFT JOIN`. I chose the `LEFT JOIN` primarily to ensure correctness regarding NULL handling. If the `Order` table's `customer_id` column contained any NULL values, a `NOT IN` subquery would unexpectedly evaluate to UNKNOWN and fail to return the correct list of customers. The `LEFT JOIN` safely avoids this logical trap and is often optimized highly efficiently by modern database execution engines.

Normalizing the Enrollment table to 3NF successfully eliminates data redundancy and prevents modification anomalies, but inherently reduces query convenience. Once split into four distinct tables, answering a basic question—like finding the instructor

name for a specific student—requires writing complex `JOIN` operations across the `Student`, `Enrollment`, `Course`, and `Instructor` tables. Real-world database systems might intentionally denormalize data—such as keeping the `course_name` or `instructor_name` directly inside the `Enrollment` table—to avoid expensive table joins for read-heavy analytical workloads. This trade-off intentionally prioritizes faster read performance and simpler reporting queries over strict storage efficiency.